

ELTE EÖTVÖS LORÁND UNIVERSITY

IMPLEMENTING WEB SERVICES

Modern web application development in .NET

Lecture 4

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

WEB SERVICES

A *web service* is a set of protocols and rules that allow platform-independent data exchange between applications over a network

- i.e. a *service provider* provides an interface of functions that can be accessed by *service consumers*
  - a consumer can be any application, e.g. a web page, a desktop application, another webservice
- a variety of protocols and solutions can be used for communication, e.g. *SOAP (Simple Object Access Protocol)*, *WSDL (Web Services Description Language)*, or *REST (Representational State Transfer)*
- allows the creation of a *Service Oriented Architecture (SOA)*

REST

REST (Representational State Transfer) is a software architecture that enables the development of scalable, high-performance distributed network applications

- communicates primarily on HTTP basis using basic HTTP methods (**GET**, **POST**, **PUT**, **DELETE**, ...)
- applications following the REST architecture are *RESTful* apps

Defines constraints to the system that must be followed:

1. client-server model,
2. uniform interface,
3. stateless communication,
4. layered architecture,
5. cacheability,
6. code on demand.

ASP.NET WEBAPI

ASP.NET Core MVC enables the development of RESTful applications

- implemented in an MVC architecture, activities are controlled by *controllers* that respond to a given request and resource
- the same Microsoft.NET.Sdk.Web SDK can be used for development, same as for the previous MVC web applications
- data in communicated in *JSON (Javascript Object Notation)* format by default, but the web service can automatically modify the format according to a client's request
- can be easily integrated into web pages developed in the ASP.NET Core MVC framework, a single web project can provide a classic web page and a web service interface as well

JSON

- JSON is a simple format for representing objects, e.g.:

```
{ // object
  "id": 1234, // property
  "group": "tool",
  "name": "hammer",
  "responsible": { "name" : "John" },
                  // composite property
  "materials": [ // array property
    { "name": "steel" },
    ...
  ]
}
```

CONTROLLERS

Controllers (still derived from the **`Controller`** class) implement how the HTTP action methods (**`GET`**, **`POST`**, ...) should be served

- on successful execution, the return value is placed in the body of the HTTP response, and an **`OK`** (200) response is generated
  - if there is no return value (void), a **`No Content`** (204) response is sent (without body)
- the operations are resolved according to the path mapping by the *routing engine*, typically in the following form:
  `<domain>/api/<controller>/<parameters>`
  - overloading actions is limited (notice there is no action in route)
  - in addition to the resource address, content may be placed and sent in the body of the request (**`FromBody`**)

CONTROLLERS

- Action methods should still return an **`IActionResult`**
  - since (primitive or complex) objects will be returned, an **`ObjectResult`** should be returned as an implementation
  - the **`Controller`** base class providers helper functions to achieve this, e.g. **`Ok(obj)`** will return an **`ObjectResult`** with the proper status code (200) and body
- For easier syntax action methods can have any return type in a WebAPI project and an object of that type can be returned
  - ASP.NET Core will automatically wrap it into an **`Ok()`** call
- Responses can be still be generated asynchronously (**`Task<IActionResult>`**, **`Task<T>`**)

CONTROLLERS

- e.g.:

```
[Route("api/myproducts")]
public class ProductsController : Controller {
  IList<string> products; // modell
  ...
  // GET /api/myproducts/
  [HttpGet]
  public IEnumerable<string> Get() {
    return products; // return all products
  }
  // GET /api/myproducts/1
  [HttpGet("{id}")]
  public string Get([FromRoute] int id) {
    return products[id]; // return specific product
  }
```

CONTROLLERS

- e.g.:

```
    // POST /api/myproducts/
    [HttpPost]
    public void Post([FromBody] string product) {
      // specify to load the product from the body
      products.Add(product);
    }

    // DELETE /api/myproducts/1
    [HttpDelete("{id}")]
    public void Delete([FromRoute] int id) {
      products.RemoveAt(id);
    }
  }
```

ROUTING

The ASP.NET Core WebAPI application must be properly configured with path resolution rules before usage

- routing is configured usually not globally like for MVC web pages, but at the controller and action level with *routing attributes*
- e.g.:

```
[Route("api/myproducts")]
public class ProductsController : Controller {
  // GET /api/myproducts/1
  [HttpGet("{id}")]
  public string Get(int id) {
    // …
  }
}
```

DATA TRANSFER

Web service operations can communicate through not only primitive types, but also ith complex types, *Data Transfer Objects (DTOs)*

- a DTO can be any object that is serializable (in the expected format), i.e. mappable to a structure of primitive values
- its structure must be chosen in such a way that it does not contain internal data to the web service, or unnecessary or circular references (e.g. in the case of an entity object)
- DTO serialization is mostly done automatically by ASP.NET Core
    - we may return a uniquely configured HTTP message (**`ContentResult`**) when needed

DATA TRANSFER

- e.g.:

```
public class Product { // DTO type
  public int Id { get; set; }
  public string Name { get; set; }
}

[Route("api/myproducts")]
public class ProductsController : Controller {
  // GET /api/myproducts/
  [HttpGet]
  public IEnumerable<Product> Get() {
    return products; // return all products
  }
  ...
}
```

DATA TRANSFER

- e.g.:

```
[Route("api/myproducts")]
public class ProductsController : Controller {
  ...
  // elérés GET /api/products/
  public IActionResult Get() {
    return new ContentResult() {
      // custom response, set the status code and the content
      StatusCode = HttpStatusCode.OK,
      Content = JsonConvert.SerializeObject(products);
                // string
    };
  }
}
```

ROUTING

Mapping HTTP methods to controller actions be done

- automatically, based on the prefix of the action's name, e.g.:

```
[Route("api/myproducts")]
public class ProductsController : Controller {
  ...
  // GET /api/myproducts
  public Product GetAllProduct() { ... }
  ...
  // DELETE /api/myproducts
  public void DeleteAllProducts() { ... }
  ...
}
```

ROUTING

- manually, applying attributes to specify the path (Route) or even the wanted HTTP method (HttpGet, HttpPost, HttpDelete, …), e.g.:

```
public class ProductsController : Controller {
   ...
   // GET /api/myproducts/
   [Route("api/myproducts/{id}")]
   public Product GetProduct(int id) { ... }
   ...
}
```

ROUTING

- routing rules defined on the controller and on the action method can also be combined:

```
[Route("api/myproducts")]
public class ProductsController : Controller {
  ...
  // GET /api/myproducts/1
  [HttpGet("{id}")]
  public Product FindProduct(int id) { ... }
  ...
}
```

- usage of controller classes must be registered in the **`Program`** class: **`builder.Services.AddControllers()`**
  - it was similar for web pages, where views were also registered: **`builder.Services.AddControllersWithViews()`**

ROUTING

When constructing the routing rules, it is supported to:

- add a prefix at the controller level (**`Route`**)
- delimit the parameters in any preferred way (by adding additional route components)
- add constraints to parameters based on their type (bool, **`datetime`**, **`decimal`**, **`double`**, **`float`**, **`guid`**, **`int`**, **`long`**), length (**`length`**, **`minlength`**, **`maxlength`**), value (**`min`**, **`max`**, **`range`**, **`values`**), or format (**`alpha`**, **`regex`**)
- specify the priority (**`Order`**) if the route matches multiple methods

Parameter type specification allows overloading, as the system can run the operation corresponding to the type.

ROUTING

- e.g.:

```
[Route("api/myproducts")] // path prefix
public class ProductsController : Controller {
  ...
  // GET /api/myproducts/1
  [Route("{id:int:min(1)}")]
  public Product GetProduct(int id) { ... }

  // GET /api/myproducts/tools/item/1
  [Route("{group:values(tools|machines)}/
           item/{id:int:min(1)}")]
  public Product GetProduct(string group, int id) { ... }
}
```

ERROR HANDLING

Controllers can respond to requests not only with the default **`OK (200)`**, but also with an arbitrary HTTP code (the generic return **`IActionResult`** must be used as return type)

- inherited helper functions can be used in an action method to easily construct the wanted result: **`Ok(<content>)`**, **`Created(<location>, <content>)`**, **`Redirect(<location>)`**, **`NotFound()`**, **`Forbidden()`**, **`Unauthorized()`**, **`BadRequest(<message>)`**, **`Conflict()`**, **`InternalServerError(<exception>)`**
- proper feedback is also important for error handling
  - uncaught exceptions result in an **`INTERNAL SERVER ERROR (500)`** returned by the service

ERROR HANDLING

- e.g.:

```
public IActionResult GetProduct(int id)
{
  try {
    ...
    return Ok(product);
    // in case the product was found, return code 200
  }
  catch {
    return NotFound(); // otherwise return code 404
  }
}
```

API CONTROLLERS

Controller classes for web services should be annotated with the **`ApiController`** attribute, which adds a range configurations for the controller and its actions. The most important are:

- *routing rules* can only be specified with the **`Routing`** attribute, rules specified with **`app.UseEndpoints()`** in **`Program`** class are not applied
  - same as calling **`app.MapControllers()`** in **`Program`** class
- automatically validate the viewmodel of the actions and return an error message on failure - 400 (*Bad request*)
  - as if all actions include the following check:

```
if (!ModelState.IsValid) {
  return BadRequest(ModelState);
}
```

API CONTROLLERS

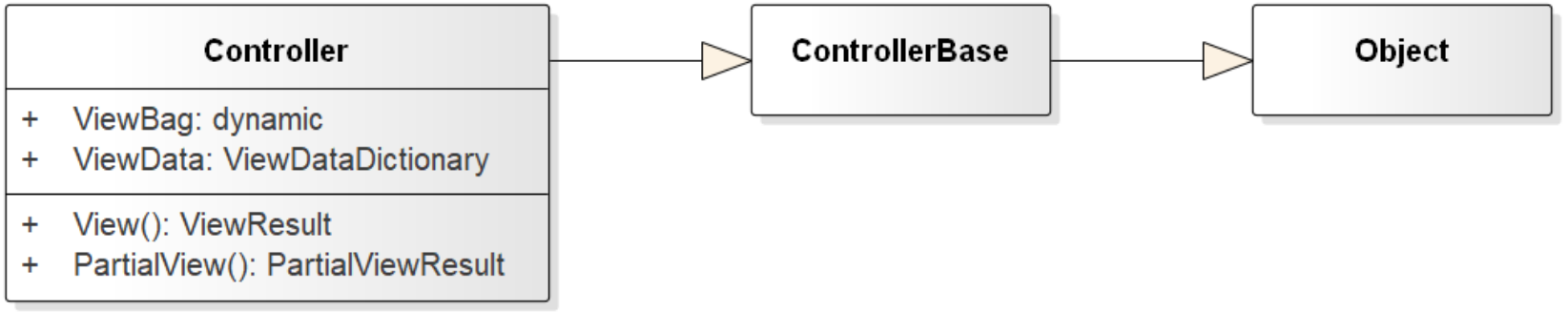
- implicitly define the source of parameters for actions, as follows:
  - **`[FromBody]`** for all complex types, with some exceptions (e.g. **`IFormCollection`**, **`CancellationToken`**)
  - **`[FromForm]`** for all parameters of types **`IFormFile`** and **`IFormFileCollection`**.
  - **`[FromRoute]`** for all parameters defined with the same name as in the routing rule.
  - **`[FromQuery]`** for all additional parameters.
- e.g.:

```
[ApiController]
[Route("api/[controller]")] // [controller] is a placeholder
public class ProductsController : Controller
```

CONTROLLER BASE CLASS

The API controller classes can be derived from the **`ControllerBase`** class instead of the **`Controller`** class.

- The **`ControllerBase`** class is the ancestor of the **`Controller`** class, and has all the necessary functionality except for support for views.



- If a single controller class serves both views and API requests, it is best to derive it from the **`Controller`** class.

OPENAPI SPECIFICATION

The *OpenAPI specification* (formerly known as *Swagger specification*) is an open format which defines the interface of a web service in JSON or YAML data serialization language, including:

- endpoints (and supported HTTP methods);
- input and output parameters;
- authentication methods used; and
- other meta information (e.g. license).

A formal specification describing a web service in the OpenAPI standard provides a unified interface that can be used by service providers and consumers to implement it.

SWAGGER

Swagger as a company provides implementation tools (some free, some commercial) for OpenAPI. Its main tools are:

- *Swagger Editor*: web editor to simplify editing an OpenAPI specification
- *Swagger UI*: web API testing interface for a developed web service.
- *Swagger Codegen*: code generator to build a skeleton implementation containing e.g. the procedure stubs, based on the OpenAPI specification of the web service. Supports a variety of languages.

Tools from other companies and also available, e.g. *OpenAPI Generator*, *Redoc(ly)*, *NSwag*.

SWAGGER UI

For web application built in ASP.NET Core MVC framework, the OpenAPI specification and a Swagger UI tester interface can be easily generated.

- Multiple NuGet packages are available, most widely used are **`Swashbuckle.AspNetCore`** and **`NSwag`**.
- The default project template in VS uses the **`Swashbuckle.AspNetCore`** package, and automatically registers the necessary generator in the **`Program`** class on the **`WebApplicationBuilder`** object:

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c => {
  c.SwaggerDoc("v1", new OpenApiInfo {
    Title = "Product Manager", Version = "v1"});
});
```

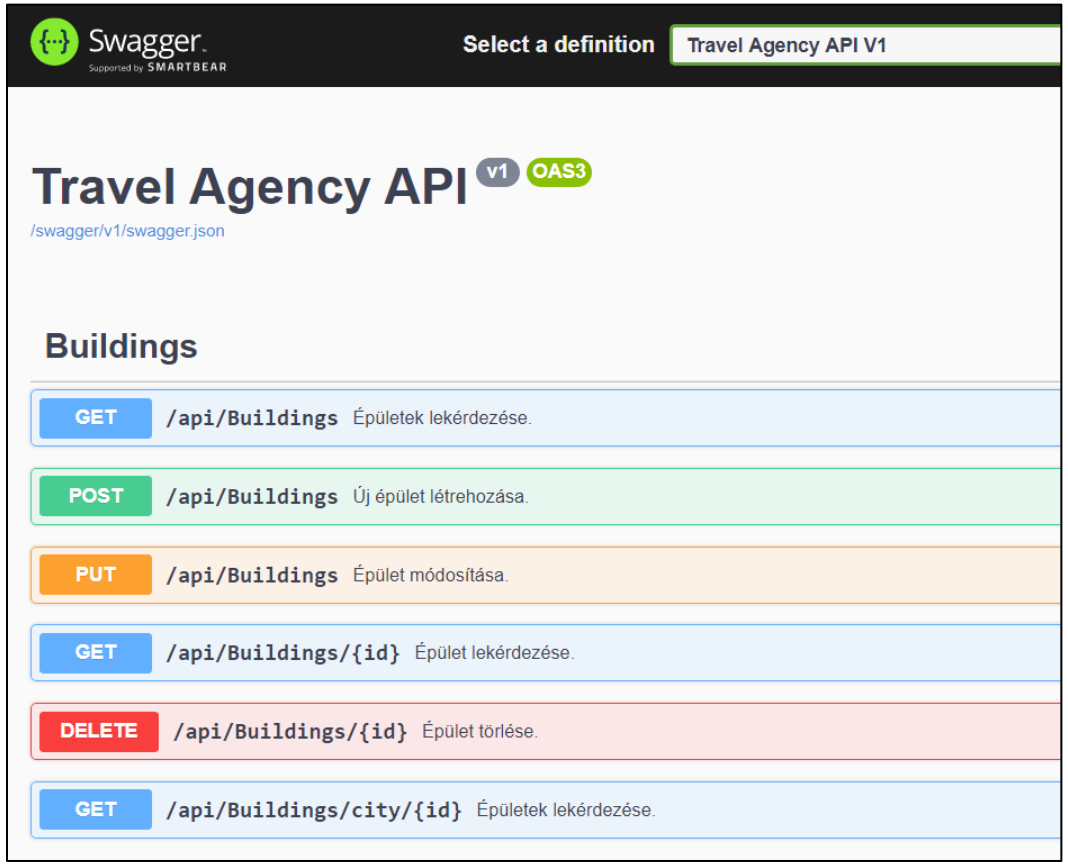
SWAGGER UI

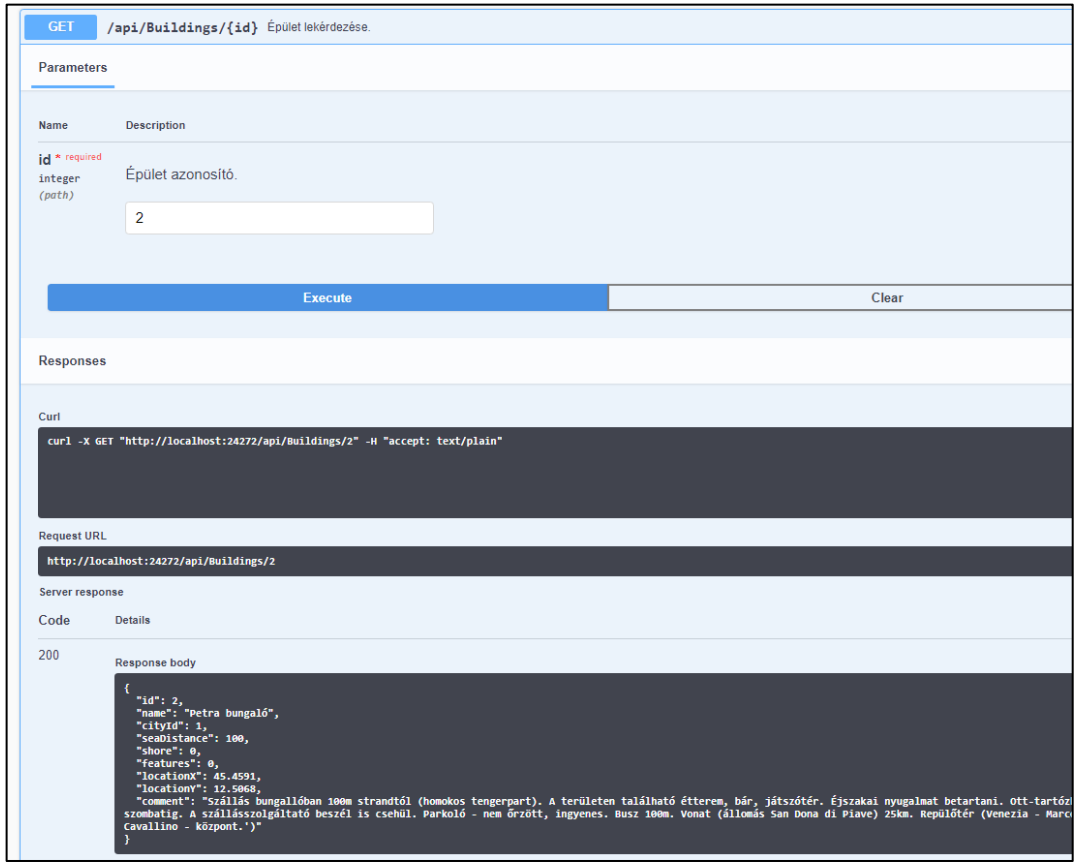
- Then Swagger can be enabled on the **`WebApplication`** object in the Program class:

```
// use Swagger (JSON endpoints)
app.UseSwagger();
// use Swagger UI (HTML endpoints)
app.UseSwaggerUI(c => {
  // configure the JSON endpoint (optional)
  c.SwaggerEndpoint(
    "/swagger/v1/swagger.json",
    "Product Manager API V1");
});
```

- Generated URLs:
  http://<domain>/swagger/index.html
  http://<domain>/swagger/v1/swagger.json

SWAGGER UI


Swagger
Supported by SMARTBEAR
Select a definition
Travel Agency API V1
Travel Agency API
v1
OAS3
/swagger/v1/swagger.json
Buildings
GET /api/Buildings Épületek lekérdezése.
POST /api/Buildings Új épület létrehozása.
PUT /api/Buildings Épület módosítása.
GET /api/Buildings/{id} Épület lekérdezése.
DELETE /api/Buildings/{id} Épület törlése.
GET /api/Buildings/city/{id} Épületek lekérdezése.


GET /api/Buildings/{id} Épület lekérdezése.
Parameters
Name
Description
id * required
integer
(path)
Épület azonosító.
2
Execute
Clear
Responses
Curl
curl -X GET "http://localhost:24272/api/Buildings/2" -H "accept: text/plain"
Request URL
http://localhost:24272/api/Buildings/2
Server response
Code
Details
200
Response body
{
"id": 2,
"name": "Petra bungaló",
"cityId": 1,
"seaDistance": 100,
"shore": 0,
"features": 0,
"locationX": 45.4591,
"locationY": 12.5068,
"comment": "Szállás bungallóban 100m strandtól (homokos tengerpart). A területen található étterem, bár, játszótér. Éjszakai nyugalmat betartani. Ott-tartózk
szombatig. A szállásszolgáltató beszél is csehül. Parkoló - nem őrzött, ingyenes. Busz 100m. Vonat (állomás San Dona di Piave) 25km. Repülőtér (Venezia - Marco
Cavallino - központ.')"
}